SET 8

PART A — Short questions and answers

Q1. Describe total participation of ACCOUNT in the relationship "Customer owns Account".

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q2. Explain referential integrity between the ACCOUNT and CUSTOMER tables.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Explain an insertion anomaly in an unnormalized Customer–Account table.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Explain the purpose of the BETWEEN operator with an example on Balance.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Explain why a non-aggregated column in the SELECT list must also appear in GROUP BY.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe the purpose of an OUT parameter in a procedure.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Describe one situation in which a relational database is preferred over a document database for transactions.

Answer: Document databases allow documents to have different fields, making them useful when records vary.

Q8. Explain how MongoDB creates a collection implicitly during the first insert.

Answer: MongoDB stores records as BSON documents inside collections.

Q9. Explain what the following MongoDB query returns: db.accounts.find({AccountType: "Savings", Balance: {$gte: 50000}})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Describe the direction of the relationship in the pattern (a:Account)- [:TRANSFERRED_TO]->(b:Account).

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A commercial bank maintains a Banking and Transaction System to store customer details, account

1(a) ER design:

• **CUSTOMER**(CustomerID, CustomerName, City, Phone); primary key: **CustomerID**.

• **ACCOUNT**(AccountNo, CustomerID, AccountType, Balance, BranchName); primary key: **AccountNo**.

• **TRANSACTIONS**(TransID, AccountNo, TransDate, Amount, TransType); primary key: **TransID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE CUSTOMER (
  CustomerID NUMBER PRIMARY KEY,
  CustomerName VARCHAR2(100),
  City VARCHAR2(100),
  Phone VARCHAR2(100)
);
CREATE TABLE ACCOUNT (
  AccountNo VARCHAR2(100) PRIMARY KEY,
  CustomerID NUMBER,
```

```
  AccountType VARCHAR2(100),
  Balance NUMBER,
  BranchName VARCHAR2(100)
);
CREATE TABLE TRANSACTIONS (
  TransID NUMBER PRIMARY KEY,
  AccountNo VARCHAR2(100),
  TransDate DATE,
  Amount NUMBER,
  TransType VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM TRANSACTIONS x
JOIN CUSTOMER a ON x.CustomerID = a.CustomerID
JOIN ACCOUNT b ON x.AccountNo = b.AccountNo;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. Graph database (Neo4j/Cypher): Create the required node labels and relationship types exactly as specified in the set. Example pattern:

```
CREATE (a:EntityA {id: 1, name: 'Name A'});
CREATE (b:EntityB {id: 2, name: 'Name B'});
MATCH (a:EntityA {id: 1}), (b:EntityB {id: 2})
CREATE (a)-[:RELATED_TO]->(b);
MATCH (a:EntityA)-[:RELATED_TO]->(b:EntityB)
RETURN a.name, b.name;
```